DECK WebTransport: a new medical web imaging alternative to DICOMweb for complex air networks

Jacques Fauquex
Opendicom (Jesros S.A.)
Montevideo, Uruguay
jacquesfauquex@opendicom.com

***Abstract*— The original DICOM [3] and its DICOMweb [4] evolution are a fundamental stumbling block in the engineering required to optimize the transmission of huge studies in complex air networks (mixed 5G/satellite/WIFI). Consequently, we created a new representation of DICOM objects that we called DECK (Dicom Exam Contextual Keys) and specialized a new WebTransport [5] protocol. These have the complete study, and any contextualized attribute of it as primary objects. This differs radically from the SOP instance of DICOM, and from the additional XML, JSON, HTTP, meta and bulk data of DICOMweb. We redefined both the original encoding (which becomes a flat list of key-value) and the DICOM Upper Layer (instead of which we use UDP duplex stream and asynchronous push of datagrams). REGEX [6] is used by us for attribute selection in the request to be executed on the server, so that we do not need XPATH [7] applied on a memory DOM representation of a SOP instance. Our response format is a very simple format, much more so than JSON, XML or DICM.**

These features were designed to facilitate javascript interaction with an HTTP/3 server from a web page running on the client. Such workspace gets encryption and complex network routing for free thanks to the QUIC engine of HTTP/3 WebTransport. This new DICOM communication stack provides the necessary building blocks to design a reactive mobile imaging workstation for medical web imaging.

Keywords—Webtransport, DICOM, DICOMweb

I. INTRODUCTION

Telemedicine is on the political agenda of health departments. Medical web imaging is an obvious candidate for this revolution. Better interpretation and lower costs can be achieved when very specialized radiologists report as much complex and large studies as possible in their field of expertise from wherever they might be at any moment.

At least one historical factor, but also another one relative to technology and quality constant improvement, both, slow down the deployment of medical web imaging:

- The universal standard DICOM (Digital Imaging and Communication in Medicine) has not been designed for complex networks characterized by 5G/satellite routing variability, out of order packets and datagram incidental drop. Though these are merely incidents of no intrinsic critical importance in comparison to the extraordinary recent improvements in reach, availability and capacity of internet, they may affect latency, which can pass instantly from acceptable to too bad during the communication of large imaging studies.
- Imaging studies are larger and larger over time. Many gigabytes for one study is not unusual nowadays. This means more communication time and higher risk of incident in the course of a transmission.

We evaluated that the original DICOM and its DICOMweb evolution are a fundamental stumbling block in the engineering required to optimize the transmission of huge studies in complex networks. Consequently, we created :

- an alternative evolution of DICOM, enabling discrete access to any attributes individually based on a strict key-value format,
- a protocol using WebTransport UDP duplex streams and datagram sending for communication, which fits better less stable network conditions, where some information may need to be repeated.

This is foundational work. Fruits will be harvested in future developments.

II. BACKGROUND

A. DICOM DICM

DICOM binary metadata format (DICM) extended the key/value paradigm, complementing it with multi-valued attributes and sequences of enclosed key-ordered lists of attributes. This was indeed necessary to describe DICOM complex objects of information. But as a result, parsing became difficult.

The parsing of what was thought of primarily as a serializable list of complex information makes sense only when it builds a relational database or a DOM (Document Object Model) in memory. In effect these products are indispensable for discrete access to any attribute of the root ordered list, or of enclosed ones. The DOM usually performs this with the help of a tool defined by W3C called XPATH.

B. DICOM XML and JSON

From DOM to XML serialization, there was one easy step only, and consequently a representation of DICM into XML was born and used in workstations. In fact, each DICOM toolkit (dcmtk [8], dcm4che [9], etc) defined its own XML representation. Later, the workgroup WG-27 of the DICOM standard defined the reference "NativeDicomModel" [10].

Over time, the steering wheel committee of the standard DICOM decided to make DICOM readily available on web pages, where javascript manages structures in a format called JSON when serialized. It was the occasion to define another representation [11] yet, derived form the XML one.

Both the XML and JSON representations are text-based. They replace DICM binary structuring glue by textual markup which allows to replicate :

- the association of multiple values to one key,
- the encapsulation of various items into one sequence,
- the nesting of the various attributes of a dataset into one item.

C. Incremental parsing

In DICM, XML and JSON alike, the parsing of an attribute implies the parsing of the complete context preceding it, in order to discover its chain of encapsulation. For instance, the triplet of attributes code, scheme, meaning only makes sense when related to its enclosing sequence attribute. This constraint is burdensome, especially in simple use cases. Even when the attribute should be found at the root level, previous sequences alter the classification and force the parser to dig into them before reaching the root level attribute of interest.

D. New use cases not satisfied

Furthermore, two new use cases are not well addressed by DICM, XML or JSON representations:

- Huge sets of datasets for Machine Learning. The binary is too complex; XML and JSON are too verbose and resource consuming when brought into memory as a DOM.
- Remote imaging through complex network 5G/WIFI/ satellite, in which case variable routing, and packet drop or out of order occur occasionally. In this context, the technical constraint to parse everything up to the attribute looked for has disastrous consequences. Much more information than really needed on any instant is processed, prone to the exponential risk of network incident and consequent bad latency.

III. Novelty of our "DECK" representation

Our new representation aims at incorporating the context into the key part of the attribute, so that each attribute is fully defined individually by the key. It differs radically from the other representations on these characteristics:

- Unique container for all the instances of the study.
- Direct key access to any attribute of any instances of any series of the study.

We call the new representation "Dicom Exam Contextualized Key" (DECK).

DECK can be translated back and forth from and to the already existing DICM, XML and JSON representations of DICOM datasets. It has been designed to easily serialize to any one of the three others.

IV. Methodology 1: Implementation of the Attribute "Ascending Order" Rule

In DICOM representations, the attributes shall be ordered in tag ascending order within the base dataset (and also within any encapsulated datasets).

A tag in its binary form is a sequence of two two-bytes words (group and unit). The order of the bytes within the two-bytes words used to depend on the endianness of the computer. A "big endian" word is written with the most significant bytes first. This corresponds to the plain english order where, thousands are first on the left, hundreds next, tens next and finally on the right end the less significant units. But Intel processors ordered the bytes of the word the other way round, starting with less significant units and ending with more significant thousands on the right. This is the "little endian order". DICOM used to provide two alternative transfer syntaxes to accommodate separately the little and big endian processors. No longer. Big endian has been deprecated. Little endian applies now to both 2-bytes words of the binary representation of a tags. The resulting four bytes sequence looks like that:

- byte 0 : group less significant byte (g)
- byte 1 : group most significant byte (G)
- byte 2 : unit less significant byte (u)
- byte 3 : unit most significant byte (U)

This makes ordering tags difficult, because it implies permuting bytes 0 and 1, and also 2 and 3 in each of the tags before ordering them.

Such incoherent serialization is the consequence of the industry shifting from big endian to little endian CPUs, following the impulse of Intel for the development of personal computers. But the deprecation of big endian in DICOM was a bad move, because now big endian ARM CPUs are becoming again more important than little endian architecture ones. The deprecation was also absurd, because textual representations of DICOM (XML, JSON) represent the tag by means of a chain of 8 hexadecimal chars (two consecutive ones for one byte) which mimics the big endian GgUu order.

Driven by these considerations, our internal model of a tag is a chain of 4 bytes ordered "big endian" GgUu.

V. Methodology 2: Key Format

In our binary key format PF(SI)*AR, the key is composed of two or more groups of 8 bytes, which is a nice size for 64 bit computing. We name the first group PF (like in PreFix). The last group is named AR (like in Attribute+Representation). In the interval zero or more SI (Sequence of Items) groups may be inserted.

A. Sequence of Items

A list of attributes (A) is called an item (I). A special attribute type sequence (S) contains a positional list of items. I1 comes before I2, which comes before I3 and so on.

B. Sequence containment recursion

Sequence containment recursion (SI)* is authorized.

When the attribute is buried into one or more levels of encapsulation of sequence, a chain of sequence-item is required to locate it fully.

C. Sequences and items delimiters

With the purpose of simplifying the serialization into binary DICM, we also materialize each item start, item end and sequence end of the dataset as if they were attributes. To make it possible, we created 4 private representations VR:

- VR 0x0000: S start tag (which replaces "SQ")
- VR 0x2B2B: I start tag (in ASCII would be seen ++)
- VR 0x5F5F: I stop tag (in ASCII would be seen --)
- VR 0xFFFF: S end tag

Example: Sequence with an empty item and an item with contents :

```
GGggUUuu 00000000
GGggUUuu 00000001 00000000 2B2B0000
GGggUUuu 00000001 FFFFFFFF 5F5F0000
GGggUUuu 00000002 00000000 2B2B0000
GGggUUuu 00000002 GGggUUuu 44410000
GGggUUuu 00000002 FFFFFFFF 5F5F0000
GGggUUuu FFFFFFFF
```

D. Attribute+Representation

In order to interpret correctly an attribute (A), its representation (R), including value representation and charset, needs to be known. we make this information available in 4 bytes :

- two ascii letters VR (value representation) datatype
- an uint16 index of the charset defined in attribute (0008,0005) of the dataset. The catalog of charsets is defined here [12].

We label AR (8 bytes long) to the attribute tag followed by representation details.

E. Prefix

Root tags in the dataset of a DICOM SOP instance are not prefixed by any item number. The DICOM SOP instance is the list content of a non existent root item. In the XML representation, such root item is mandatory per XML rules. This was the occasion to create the root XML element <NativeDicomModel>, unique root to the XML representation.

In DECK, the object described is an Exam, that is, a group of SOP instances. In order to identify the SOP instance referred to by a key, we prepend to the key an 8 bytes prefix. It would be conceptually equivalent to a <NativeDicomModel SOPinstance=""> in the XML representation. The prefix both gathers the attributes of a specific SOP instance and differentiates SOP instances.

We carefully crafted the format of the prefix PF. In DICOM object model, instances (and series too) are identified by both an ordinal number (the original identification) and an UID (the new universal identifier, which conforms to the syntax of the OID). The UID is defined as universally unique, but is quite large (up to 64 bytes). The number is short, but may not be unique within the study. It is possible to observe on occasions SOP instances with the same number and a different UID. We created a byte index to short numbers within the prefix to solve this rare use case. This led as to the following prefix format PF: "sSSiIIFF"(s series index, SS series number, i instance index, II instance number, FF frame number).

The PF format was further refined with a subtlety taking advantage of the special value 0 of these index to signal an attribute common to all the instances of a series or of a study so that we can de duplicate these common attributes and register them once only:

- s=0, study attribute, common to all instances
- i=0, series attribute, common to all the instances of the series
- FF=0, instance attribute, common to all the frames of the instance.

VI. METHODOLOGY 3: UDP WEBTRANSPORT

A recent evolution of HTTP/3 named WebTransport service, enables the web client to open UDP private streaming channels, vectorized or duplex with the server.

Analyzed in the perspective of the OSI (Open Systems Interconnection) stack, we observe that this is radically different than TCP, on which HTTP DICOMweb is built. In the case of DICOMweb, TCP transport and session are intertwined because of the three way handshake and HTTP headers. This consumes datagrams forth and back before transporting real data. The presentation layer then adds superstructure to transport the data (MIME, XML, JSON), and finally the application layer adds more formalism by means of strict HTTP API specification. Each new request implies instantiation of all these layers, with datagrams sent and received for initial handshake and security purposes.

For sure, session and security are also present in HTTP/3, but the difference is that thanks to the QUIC engine, the session may become permanent, enabling the continuous transport of datagrams in both directions via UDP, without repeating the initialization and finalization phases time and again. This service is called WebTransport. QUIC even manages to continue the streams existence when the device switches for instance from 5G to wifi. The only superstructure imposed is that of the datagram. Request and response datagrams can contain any binary contents. It's the user responsibility to organize it. A thoughtful organization can make this transport orders of magnitude faster than the precisely organized TCP/HTTP, which privileges multi datagrams files structures.

We built on this new client service, WebTransport, to communicate DICOM attributes between server and client, based on UDP requests of the client, (or in the form of notifications as well, not described here). Two protocols were defined:

- the fastest, let's call it GET, which implements request-response, limited to one datagram each, on the duplex stream opened from the client side.
- the size-unlimited datagrams described as PUSH, which implies a request-response on a duplex stream, and the following asynchronous datagrams push from the server to the client. The maximum value size pushed for one request in this case is 65536 x 65000 bytes, which is slightly less than 4 GB.

We applied these basic protocols with the creation of 4 primitive commands:

A. Get a value for a key

Get value for a key "GETVALUE" fetches the value corresponding to a key sent over UDP duplex stream from the client to the server.

Request:

- 1 bytes: Command: 0x40-0x4F. The last 4 bits define the priority. A lower number means higher priority.
- 1 bytes: Study Index: 1-255
- 2 bytes: Key Length (KL)
- KL bytes: Key

Response: By means of the same canal duplex, the server responds with the value, non processed, exactly as in the DICM representation. We register values as are in the binary DICM representation (including padding), that is as a byte chain. Our parsing is lazy. Value parsing is differed until representation needs it. This optimizes the parsing process, since many attributes will never be represented and don't need to be parsed. Another benefit of lazy parsing is that the serialization of the values to DICM representation is merely a copy operation.

Limitations:

- GETVALUE may query any individual attribute, except the sequences, which are made of items lists of attributes.
- If the value length were to overflow the size of a datagram, the client will be invited to use another function to get the result.

For both exceptions, the answer shall be a byte 0x7. The client then may issue a "PUSHVALUES" command with the same key (which also works as regex filter).

B. *Get a a list of keys*

The key format opens access to any attribute of any SOP instance of any series of the study, individually. But previously to getting a value, a discovery of the keys of the attributes present in a study, or subgroup of it, may be necessary. As a benefit of the careful design of the keys and the ordered list of them, this can be realized with the technique of regex selection or of shared prefix:

- regex selection filters the keys based on a sophisticated and powerful analysis of its contents;
- shared prefix selection, which is a simple case of regex selection, filters the keys which start with some prefix (of any length). Shared prefix selection is very effective for the selection of a series, an instance of a series, groups of instance (group 0002, private groups, etc), a particular sequence within a SOP instance. Also works for encapsulated sequences, or for a particular item of a particular sequence.

These techniques are similar to XPATH for XML, but work much faster, and on binary data. In the case of our API, it requires a byte capable REGEX engine, like for instance the Rust one [13].

Request: Get a list of keys, "GETKEYS" is similar to GETVALUE, but with another command-priority number, and a regex filter instead of a key parameter:

- 1 bytes: Command: 0x50-0x5F. The last 4 bits define the priority. A lower number means higher priority.
- 1 bytes: Study Index: 1-255
- 2 bytes: Regex Length (RL)
- RL bytes: REGEX which selects a group of Keys

Response: The key list response format is a sequence of length-and-key binomials:

- 1 byte: length of the key (KL)
- KL bytes: the key

All the keys are in ascending order, without any bytes before, between or after them. The list is terminated by 1 byte with value 1 (list complete).

C. *Get values*

Get values, "GETVALUES", behaves like GETKEYS, except that the result takes the form of a serialization of zero or more key-values. If the serialization fits into a datagram, it is returned. Else 0x7 is returned.

Request: Like GETKEYS, but with another command-priority number:

- 1 bytes: Command: 0x60-0x6F. The last 4 bits define the priority. A lower number means higher priority.

Response: The key-value list response format is a sequence of zero or more quadrinommal key-value structure in key ascending order, without any bytes before, between or after them. The 4 components are:

- 1 byte: length of the key (KL)
- KL bytes: the key
- 4 bytes: length of the value (VL)
- VL bytes: the value, exactly as in the DICM representation.

The list is ended with 1 byte: 1=complete-

D. *Push values*

Push values, "PUSHVALUES", behaves like GETVALUES, except that it returns:

- or a one byte error code
- or one byte "1", followed by two bytes indicating the total of datagrams to be sent

and then starts to push the corresponding datagrams.

Request: Similar to GETKEYS and GETVALUES, but with another command-priority number:

- 1 byte: Command: 0x70-0x7F. The last 4 bit define the priority. Lower number means higher priority.

Response: additionally to the short response already defined, the datagrams then pushed conform to the following format. Figure [1] "datagrams pushed":

- a 4 bytes length of continued value CL. if CL is not zero, it is followed by the continued value.
- zero or more quadrinommal key-value structure in key ascending order, without any bytes before, between or after them like in the key-value list response format.
- 1 byte with value 1=complete, or 2=value to be continued, or 3=list to be continued.

VII. DISCUSSION

The execution of the first three commands is almost instantaneous, since they involve one datagram in both directions, without any handshake. GETVALUE is new in DICOM ecosystem and is made possible by DECK Key format access to any single attribute. GETKEYS is also new, for the same reason. GETVALUES is similar to "metadata" in DICOMweb, but incomparably more flexible, thanks to the REGEX filter applied to the complete study. In particular, it fits very well the use case of getting a specific set of discrete attributes. An extension, or new commands could refer to pre-configured standard sets.

The response to the forth command, PUSHVALUES, is absolutely different from DICOMweb concepts, but similar to the original DICM DIMSE communication, where a C-MOVE command initiates server side asynchronous push of packets. Compared to the later, PUSHVALUES:

- is optimized for web datagrams
- functions inside an UDP channel automatically created between client and server
- is both broader (since it reaches all the instances of a study) and fine-tuned thanks to the REGEX filter.

VIII. CONCLUSIONS

DECK encoding and WebTransport basic commands were designed to solve the new use cases of machine learning and air networks:

- huge sets of datasets for machine learning can be traversed study by study, focusing on the attributes of interest only.

- remote imaging workstation through complex network 5G/WIFI/satellite, is resolved by the protocol HTTP/3 WebTransport. The volume of data to be transported can be surgically reduced, and missing datagrams pushed again individually. The integrity control performed on UDP datagrams can be optimized in comparison to TCP data control, which is file-based. DECK is perfect for the building of client reactive web pages.

Being a new way to transport DICOM, specifically for consumption from a web page using javascript through air networks (5G, satellite, wifi) connecting to a HTTP/3 server, DECK WebTransport might qualify for a definition as a supplement to the DICOM standard as a "new Application Entity" figure [2]. In this paper, Service Class Specifications where defined for querying and fetching. Other ones would be expected to be implemented too, like storing.

In the long run, thanks to its dependence on HTTP/3 WebTransport, DECK WebTransport might become more important than DICOMweb. It just fits better air networks, which are almost by definition the main networks for medical web imaging. A theoretical metrics (the number of datagrams sent) to realize a specific visualization advocates clearly for the benefits of DECK WebTransport over DICOMweb. The capacity of HTTP/3 to switch between internet source without interruption of the duplex stream is another huge advantage. Finally, the simplified parsing reduces even more latency.

This work will be completed by another paper, to be written, called "HTJ2K WebTransport tiles-datagrams", where the exact same WebTransport datagram push is used to optimize the communication of pixels in the format High-throughput JPEG 2000 [14].

Acknowledgment

Thanks to the Núcleo de Ingeniería Biomédica, Universidad de la República, Montevideo, Uruguay for the academic space where, lectures after lectures, the concepts exposed here consolidated over time; and to Dra Teresa Lin, who led me to be involved with the challenges of medical web imaging.

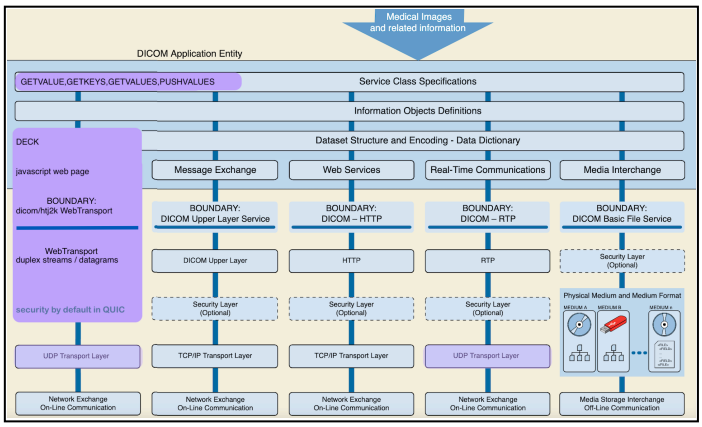
References

1. Figure 1. "*datagram pushed*"

push values datagram

Offset		octet 0								octet 1								octet 2								octet 3							
octet	bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	*Source Port*																*Destination Port*															
4	32	*Length*																*Checksum*															
8	64	*Datagram Total*																*Datagram Index*															
12	96	*Command /Error Number*								*Study Index*								*Regex Key Length*															
16	128	*Regex Key (common to all the keys)*																															
		PUSHKEYVALUES response format																															

2. Figure 2. "new Application Entity"



3. https://www.dicomstandard.org/

4. https://www.dicomstandard.org/using/dicomweb

5. https://www.w3.org/TR/webtransport/

6. https://www.gnu.org/software/findutils/manual/html_node/find_html/Regular-Expressions.html

7. https://www.w3.org/TR/xpath/

8. https://dicom.offis.de/dcmtk.php.en

9. https://web.dcm4che.org/

10. https://dicom.nema.org/dicom/2013/output/chtml/part19/chapter_A.html

11. https://dicom.nema.org/medical/dicom/current/output/chtml/part18/sect_f.2.html

12. https://github.com/jacquesfauquex/DECK/wiki/parser-repertoires

13. https://docs.rs/regex/latest/regex/bytes/index.html

14. https://www.itu.int/rec/T-REC-T.814-201906-I